MAKÜ
BURDUR MEHMET AKİF ERSOY ÜNİVERSİTESİ

VERİ YAPILARI VE ALGORİTMALAR

HAFTA 5
BAĞLI LİSTELER

Dr. Öğr. Üyesi Ebru AYDOĞAN

Bucak Bilgisayar ve Bilişim Fakültesi

Ders İçeriği: Hafta 5

- Bağlı Listeler
- Bağlı Liste Veri Yapısı
- Bağlı Liste İşlemleri
- Tek Yönlü Bağlı Listeler
- İki Yönlü Bağlı Listeler
- Dairesel Bağlı Listeler

Bağlı Listeler

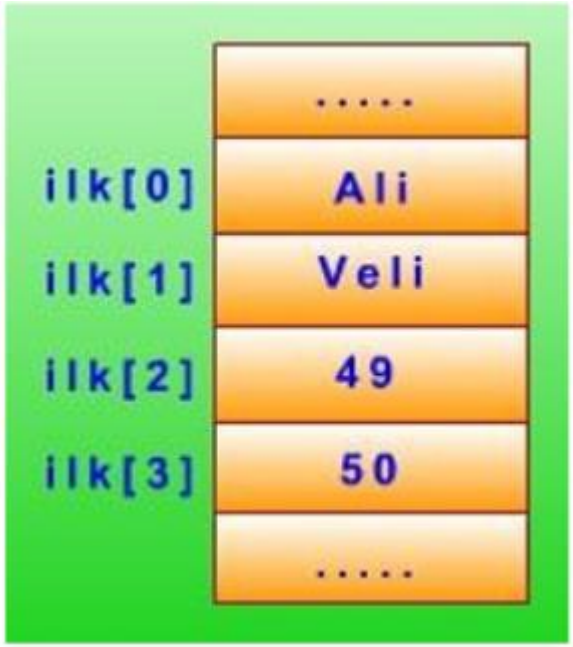
- Bellekte elemanları ardışık olarak bulunmayan listelere bağlı liste denir.
- Bağlı listelerde her eleman kendinden sonraki elemanın nerede olduğu bilgisini tutar.
- İlk elemanın yeri ise yapı türünden bir göstericide tutulur. Böylece bağlı listenin tüm elemanlarına ulaşılabilir.
- Bağlı liste dizisinin her elemanı bir yapı nesnesidir.
- Bu yapı nesnesinin bazı üyeleri bağlı liste elemanlarının değerlerini veya taşıyacakları diğer bilgileri tutarken, bir üyesi ise kendinden sonraki bağlı liste elemanı olan yapı nesnesinin adres bilgisini tutar.

Bağlı Listeler

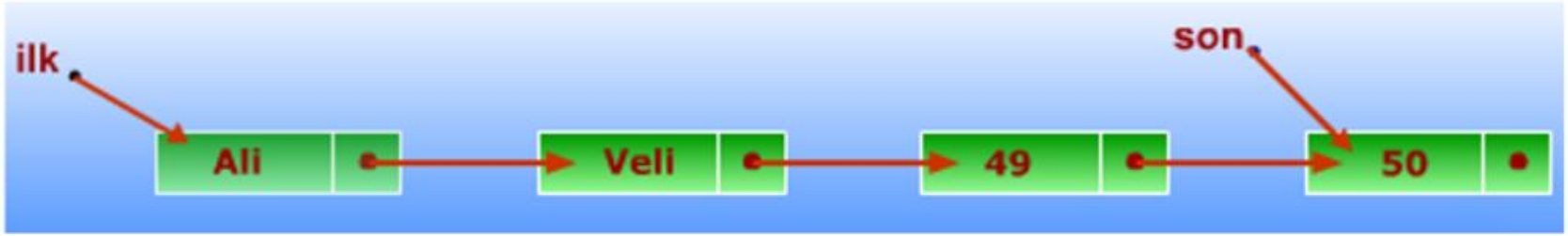
- Doğrusal veri yapılarında dinamik bir yaklaşım yoktur. İstenildiğinde bellek alanı alınamaz ya da eldeki bellek alanları iade edilemez.
- Bağlı listeler dinamik veri yapılar olup yukarıdaki işlemlerin yapılmasına olanak verir.
- Bağlı listelerde düğüm ismi verilen bellek büyüklükleri kullanılır.
- Bağllı listeler çeşitli tiplerde kullanılmaktadır;
  - Tek yönlü doğrusal bağlı liste
  - İki yönlü doğrusal bağlı liste
  - Tek yönlü dairesel bağlı liste
  - İki yönlü dairesel bağlı liste

Bağlı Listeler

- Doğrusal Liste (Dizi) Örneği



- Bağlı Liste Örneği



Bağlı Listeler

Bağlı Listelerle Dizilerin Karşılaştırılması

- Diziler;
  - Boyut değiştirme zordur
  - Yeni bir eleman ekleme zordur
  - Bir elemanı silme zordur
  - Dizinin tüm elemanları için hafızada yer ayrılır
- Bağlı listeler ile bu problemler çözülebilir.
  - Her dizi elamanı için ayrı hafıza alanı ayrılır.
  - Bilgi kavramsal olarak sıralıdır ancak hafızada bulunduğu yer sıralı değildir.
  - Her bir eleman (node) bir sonrakini gösterir.

Bağlı Liste İşlemleri

- Listeye eleman ekleme
  - Başa
  - Sona
  - Araya
- Listeden eleman silme
  - Baştan
  - Sondan
  - Tümünü
  - Belirlenen bilgiye sahip elemanı
  - İstenen sıradaki elemanı
- Arama
- Listeleme
- Kontrol

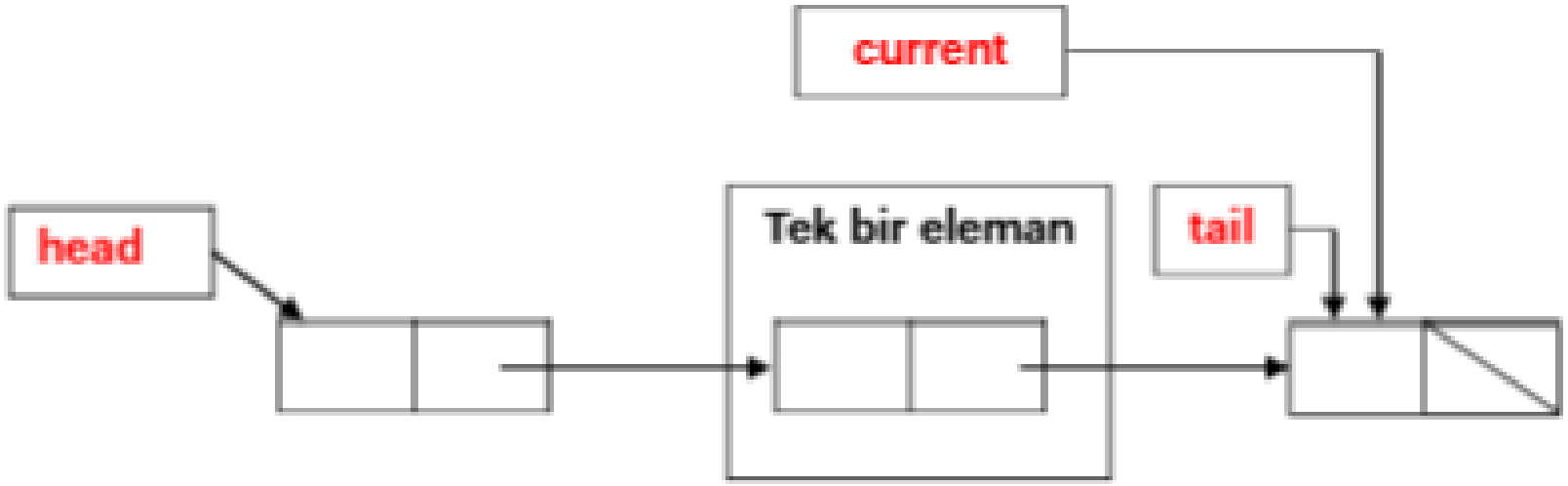
Bağlı Liste Veri Yapısı

- Listedeki her bir eleman(node, düğüm), **veri** ve **bağlantı** kısmından oluşur.
  - Veri, içerisinde saklanan bilgiyi ifade eder.
  - Bağlantı, kendisinden sonraki elamanın göstericisidir(pointer).

Bağlı Liste Veri Yapısı

- Head(Başlangıç): Listenin ilk elemanını gösterir.
- Tail(Kuyruk, Arka): Listenin son elemanını gösterir.

- Current(aktif): Listede şu anda bilgilerine ulaşabileceğimiz elemandır.



Tek Yönlü Bağlı Listeler

- Listedeki elemanlar arasında sadece tek bir bağ vardır.
- Bu tür bağlı listelerde hareket yönü sadece listenin başından sonuna doğrudur.

Tek Yönlü Bağlı Listeler

- Bağlı listenin elemanları için struct(yapı) oluşturma

```
struct eleman{
        int data;
        struct eleman *sonraki;
};
typedef struct eleman e;
```

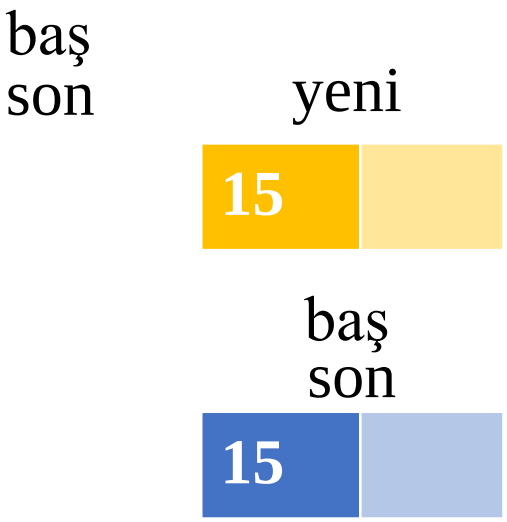
Tek Yönlü Bağlı Listeler

- Bağlı listenin başlangıç ve sonuncu elemanlarını tanımlama

```
struct eleman *bas = NULL;
e *son = NULL;
```

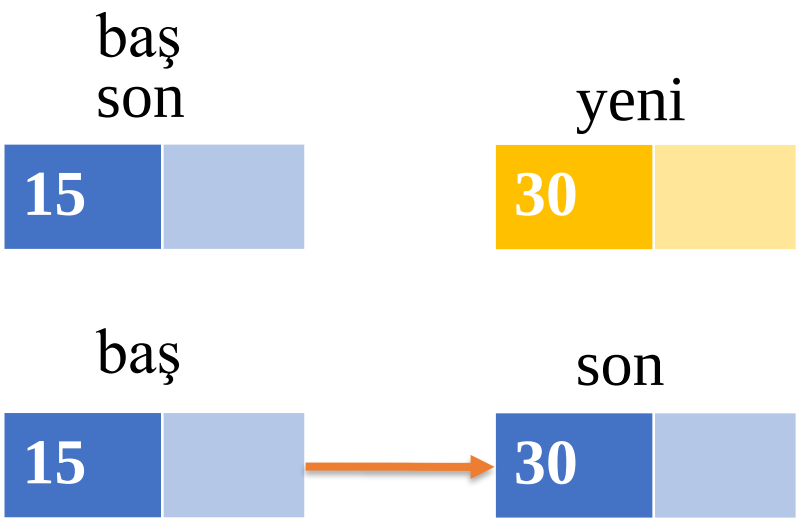
Tek Yönlü Bağlı Listeler

- **Bağlı listenin sonuna eleman ekleme**
  - Listede hiç eleman yoksa yeni eklenen eleman hem baş hem de son elemandır.
  - Elemanın işaretçisi NULL'dur



➢ Bağlı listenin sonuna eleman ekleme

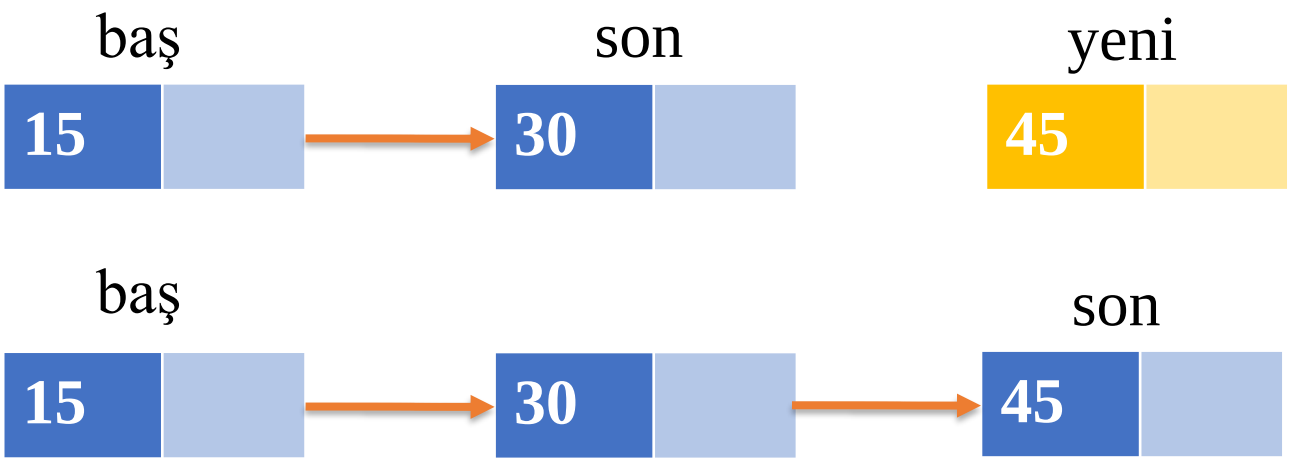
- Listede eleman varsa yeni eklenen eleman son elemandır. Baş eleman değişmez.
- Son elemanın işaretçisi yeni oluşan son elemana bağlanır.



Tek Yönlü Bağlı Listeler

➢ Bağlı listenin sonuna eleman ekleme

- Listede eleman varsa yeni eklenen eleman son elemandır. Baş elemanda değişmez.
- Son elemanın işaretçisi yeni oluşan son elemana bağlanır.



Tek Yönlü Bağlı Listeler

➢ Bağlı listenin sonuna eleman ekleme

```
// listenin sonuna eleman ekleme fonksiyonu
int sona_eleman_ekle(int veri_ekle)
{
    e *yeni = malloc(sizeof(e));    // eleman boyutunda bellekte dinamik alan açma
    yeni->veri = veri_ekle;
    yeni->sonraki=NULL;

    if(bas==NULL)  //listede hiç eleman yoksa
    {
        bas = yeni; //yeni eleman aynı zamanda baş elemandır.
    }
    else    //listede eleman varsa
    {
        son->sonraki=yeni;   //son elemanın işaretçisi yeni elemanı göstersin
    }
    son = yeni;    //yeni eleman artık son elemandır

}
```